

ESCUELA DE
INGENIERÍA INFORMÁTICA



PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO

INFORME TÉCNICO STEAM

PABLO AGUILERA - 21.712.853-6
DANIEL CORNEJO - 21.628.427-5
JOAQUÍN GARRIDO - 20.882.540-2
BENJAMIN GÓMEZ - 21.039.315-3
CRISTIAN MEJÍAS - 18.654.182-0
CRISTÓBAL RUBILAR -21.711.065-3

COMPUTACIÓN PARALELA Y DISTRIBUIDA

MAYO 2026

ÍNDICE GENERAL

Índice General	I
Lista de Figuras	II
1. Introducción	1
2. Fundamentación y teoría	2
2.1. Concurrencia	2
2.2. Posibles fallos en nodos independientes	2
2.3. Justificación de transparencia	4

ÍNDICE DE FIGURAS

1.	Representación usuario	3
2.	Representación nodo infraestructura de red	3
3.	Representación servidor	4
4.	Usuarios y servidores	5

1. INTRODUCCIÓN

La computación paralela y distribuida es una rama de la informática que se encarga de estudiar modelos y sistemas que permiten ejecutar tareas de forma simultánea o repartidas, entre distintas máquinas con múltiples unidades de procesamiento. En específico, la computación paralela busca obtener máximo rendimiento para dichas tareas dividiendo los problemas en partes que se puedan ejecutar al mismo tiempo en distintas máquinas. En cambio, la computación distribuida se enfoca en los sistemas cuyos componentes se encuentran conectados en computadoras conectadas en red, comunicándose y coordinándose mediante el paso de mensajes.

En este informe se tratará el tema, y específicamente sobre la aplicación Steam, app. creada por Valve y que es una de las más populares y grandes del mundo del gaming. Se verán las conexiones entre sus distintos servidores, como los servidores encargados del servicio de mensajería de chat, los servidores encargados de las bibliotecas de videojuegos de los usuarios, entre otros. Esto se verá reflejado en distintos modelos, tales como modelos físicos, modelos arquitectónicos y modelos fundamentales.

2. FUNDAMENTACIÓN Y TEORÍA

2.1. Concurrencia

La concurrencia en sistemas se puede manifestar en la capacidad que tiene el sistema de gestionar múltiples tareas o flujos de datos a la vez, sin que uno tenga que terminar necesariamente para que otra tarea comience. En esta sección se hablará de cómo Steam tiene procesos concurrentes y en qué funcionalidades se puede observar esto.

Desacoplamiento: El modelo de Steam se basa principalmente en conexiones uno a uno de cliente-servidor, pero incluso en estas conexiones existen procesos concurrentes. Esto se puede ver reflejado en, por ejemplo, si un usuario está ejecutando una tarea de enviar un mensaje, el servidor puede estar procesando otro mensaje que envió otro usuario, haciendo este proceso un proceso concurrente y no iterativo, ya que el segundo usuario pudo mandar el mensaje sin que tuviera que esperar a que se procese el mensaje del primer usuario. Gracias al buffer que el sistema posee, se permite que el flujo de salida del usuario y el flujo de procesamiento del servidor ocurran de forma concurrente en vez de manera iterativa o secuencial.

Concurrencia en la capa de transporte TCP/IP: El protocolo TCP/IP maneja igualmente la concurrencia, aunque de menor nivel. Mientras Steam está mandando datos, el sistema operativo, simultáneamente realiza más procesos como pueden ser el encapsulamiento de datos en segmentos, la gestión de envíos de ACKs técnicos (ACK son mensajes de confirmación) y el control de flujo de la ventana, para que no sature al receptor.

Servidor, entidad concurrente: El servidor, como sistema real usa *threads* para manejar la concurrencia de usuarios, esto hace que múltiples usuarios estén conectados simultáneamente y puedan usar funcionalidades, por cada usuario que se conecte se genera un *thread* independiente, para que el procesamiento de datos de un usuario que vaya lento no bloquee la entrada a otros usuarios que también quieran usar esas funcionalidades.

La mayoría de las funcionalidades concurrentes de Steam son gracias a la integración de un buffer que permite que se procesen datos a una velocidad equilibrada. Un buffer es un espacio de memoria física o virtual que se utiliza en servidores para almacenar temporalmente datos mientras se transfieren. Esto generalmente se ve representado como una cola, que evita interrupciones en la transmisión de la información entre dispositivos o servidores.

2.2. Posibles fallos en nodos independientes

En todos los sistemas existen fallos que pueden ocasionar que se pueda caer todo el sistema por algún nodo importante, por eso se trata de construir sistemas que tengan nodos independientes y que no conlleven que se caiga todo si es que llega a fallar. Los fallos de nodos independientes son críticos igualmente en sistemas distribuidos, porque el sistema no puede distinguir entre un proceso caído y uno lento.

Fallos en nodos de usuario: Pueden existir fallos en los nodos que serían los usuarios de Steam, estos fallos pueden ser de red o de hardware (aunque también de software). Algunos ejemplos de fallos que se pueden ocasionar son que se le vaya la luz al usuario, que el sistema operativo del usuario le de un pantallazo azul o algún fallo interno de hardware, como que se le queme el procesador u otro componente del computador.

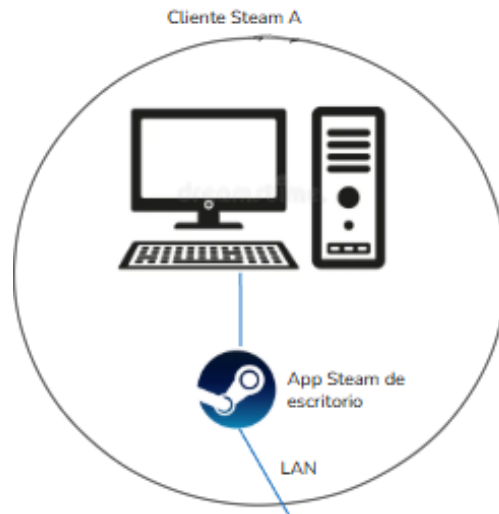


Figura 1: Representación usuario

Fallos en nodos de infraestructura de redes: Algún posible fallo que se podría identificar en la infraestructura de red es que se sobrecaliente el Router, o se vaya la luz, imposibilitando el enrutamiento del usuario. Esto tendría un impacto inmediato porque el usuario queda incomunicado con el sistema y aislado del mundo exterior.

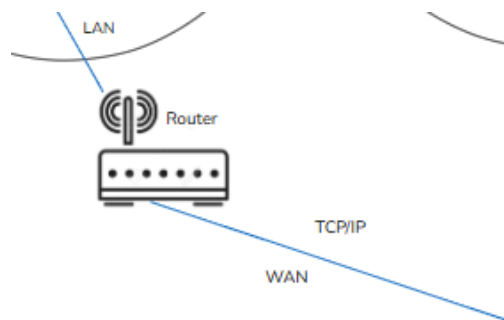


Figura 2: Representación nodo infraestructura de red

Fallos en nodos de servicio: Pueden fallar los dispositivos que componen las bases de datos de los servidores, esto generaría impactos en las funcionalidades, en temas de tiempo más que nada, ya que como se observa en la Figura 3 los dispositivos que componen la base de datos están en topología de estrella, por lo que si uno falla, los demás pueden funcionar normalmente, y así no afectaría a las funcionalidades que ofrecen. Además, el sistema cuenta con servidores de respaldo, por lo que refuerza lo que se dijo antes de que no afectaría.

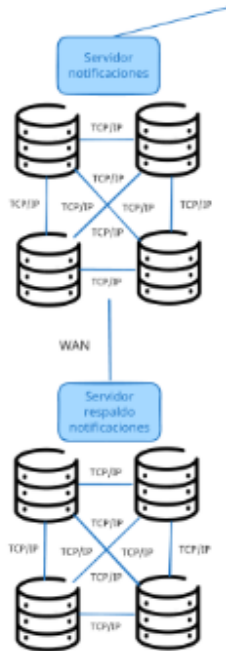


Figura 3: Representación servidor

2.3. Justificación de transparencia

Para justificar la transparencia en el sistema propuesto, es necesario analizar cómo el diseño oculta la complejidad de la red y la distribución geográfica, permitiendo que el usuario e incluso los admins interactúen con el sistema como si fuera una única entidad local.

En esta parte se detallará cómo se logran estas transparencias integrando los modelos y el contexto de los servicios de Steam.

Transparencia de acceso: Este tipo de transparencia garantiza que la forma en la cuál se puede acceder a un recurso sea la misma, esto si es remoto o local. La comunicación se centraliza en el uso de sockets como *endpoints*. La abstracción oculta si el mensaje viaja por la memoria del sistema o si va por la red. Cuando un usuario envía un mensaje al chat, la interfaz de Steam no cambia si el servidor de mensajería está en su ciudad o si está en otro lugar. La transparencia de acceso de Steam se logra porque la capa de aplicación delega en el *middleware* la tarea de hacer el *marshalling* de los datos, convirtiéndolos en un formato accesible para que el protocolo TCP/IP pueda funcionar sin que el usuario tenga que intervenir para esto.

Transparencia de ubicación: Esta transparencia permite acceder a los recursos (que pueden ser archivos, servidores, bases de datos) sin la necesidad de conocer su ubicación física (dirección IP o rack). En la Figura 4 se observa que los usuarios están separados de los servidores por múltiples saltos. La transparencia de ubicación se logra porque el cliente no necesita conocer la ruta física ni en qué puerto del switch está conectada la Base de Datos, por lo que usa identificadores lógicos o en su defecto nombres de servicio. El usuario al realizar una compra de un juego/producto, inicia un proceso síncrono (similar a un RPC), el usuario simplemente interactúa con la tienda, mientras que el sistema resuelve dinámicamente si la petición debe ir a un servidor con una zona horaria espe-

cífica para optimizar la carga, y esto mantiene la ubicación física del servidor totalmente invisible para el usuario.

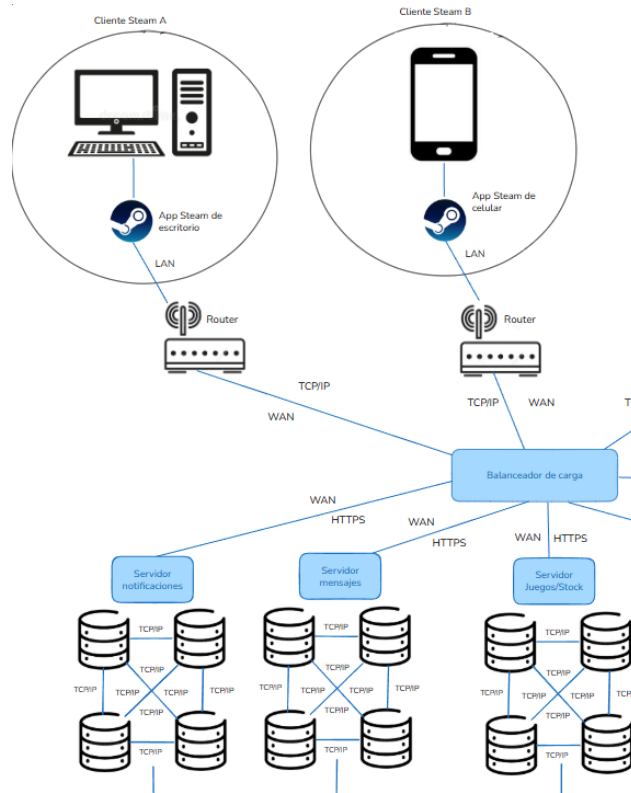


Figura 4: Usuarios y servidores

La implementación logra estas transparencias al desacoplar la lógica del negocio de la infraestructura de red. Mientras que el modelo físico define dónde están las máquinas y los firewalls, los modelos de comunicación síncronos y asíncronos definen cómo fluyen los mensajes a través de interfaces uniformes. Esto asegura que, aunque un servidor de Steam se mueva o cambie su configuración interna, el acceso y la experiencia para el usuario final permanezcan inalterados.